

PN532 NFC Controller

Hardware Overview

What is it?

The **PN532 Controller** is an NFC (*Near Field Communication*) reader and writer. It allows you to store and retrieve data—like text, numbers, or unique Serial Numbers (UIDs)—on plastic tags or cards.

Electrical Explanation

The module contains a processor and a built-in copper antenna.

- **Magnetic Field:** When powered, it emits a 13.56 MHz magnetic field.
- **Passive Power:** The tags have no batteries. When a tag enters the field, it harvests energy from the magnetic field, powers up its internal microchip, and responds to the module.

Key Hardware Facts:

- **Connection:** I²C Bus (SDA/SCL).
- **Default Address:** 0x24.
- **Voltage:** Operates at 3.3V to 5V.
- **DIP Switches:** The physical switches on the board must be set to I²C mode:
Switch 1 = ON (1), Switch 2 = OFF (0).
- **Signal Flow:** K-Comp → I2C → PN532 Chip → Antenna → Magnetic Field → NFC Tag.



Software & Programming

kcomp_NEO6MGPS.h

NFC communication may involve complex cryptography, hardware dialects, and memory protocols. This library handles the I²C polling, reading, parsing, and formatting behind the scenes.

Usage: `#include <kcomp_PN532.h>`

 **Tag Types & The "Slot" Architecture** Unlike sensors that give you a single number (like temperature), NFC tags contain an EEPROM. This library uses the concept of a **Slot**. A Slot is an abstract container that holds **exactly 16 Bytes** of data. How that slot physically maps to the silicon depends on the type of tag you are holding:

- Type A: MIFARE Classic 1K
 - Contains 1024 bytes divided into locked blocks.
 - **Danger:** Do not write to Slot 0 (contains the permanent UID) or Slots 3, 7, 11, and 15 (these contain the passwords. Overwriting them bricks the card!).
 - **Safe Slots:** 1, 2, 4, 5, 6, 8, 9, 10...
- Type B: NTAG / Ultralight
 - Contains smaller memory (e.g., 144 bytes) laid out in 4-byte pages. The library stitches 4 pages together to make a 16-byte Slot.
 - **Danger:** Do not write to Slots 0, 1, 2, or 3 (contains system formatting).

- **Safe Slots:** Begin at Slot 4. *(Note: Because 1 slot takes up 4 physical pages, you must space your data out by jumping by 4: Slot 4, Slot 8, Slot 12... otherwise you overwrite your data).*

Function Reference

Function Name	Description & Parameters
<code>bool initPN532()</code>	Setup: Initializes the controller. Returns: <code>true</code> if successful.
<code>int8_t scanForTag()</code>	The Worker: Polls the physical environment for a tag. Returns: <code>PN532_OK</code> if found, <code>PN532_NO_CARD</code> if empty.
<code>uint8_t getTagType()</code>	Read: Returns the architecture (<code>CARD_MIFARE_CLASSIC</code> , <code>CARD_NTAG</code> , or <code>CARD_UNKNOWN</code>).
<code>void getUid(uint8_t array[])</code>	Read: Copies the tag's Serial Number into your array. array: Array UID gets stored in. Must be at least 7 bytes long.
<code>int8_t readTagMemory(uint8_t slot, uint8_t buffer[])</code>	Action: Reads exactly 16 bytes from the <code>slot</code> into your buffer array. slot: Memory slot which is read. buffer: Buffer the memory slot is read in. Must be at least 16 bytes long.
<code>int8_t writeTagMemory(uint8_t slot, uint8_t data[])</code>	Action: Burns exactly 16 bytes from your data array permanently into the <code>slot</code> . slot: Memory slot which is written. data: The memory slot is filled with data. Must be at least 16 bytes long.

⚠ Pitfalls & Troubleshooting

The Wrong Dialect (DIP Switches)

- If the code crashes at `initPN532()`, the Arduino cannot physically see the sensor.
The Fix: Check the tiny switches on the module. They must be set to I²C (1: ON, 2: OFF).



Sample Program

This program allows you to read from or write to a tag using the Serial Monitor menu.

```
#include <kcomp_PN532.h>

#define TARGET_SLOT 4 // Safe user memory page

// Gets written to the EEPROM. MUST be at least 16 bytes.
uint8_t globalData[16] = "ABCDEFGHJKLMNO!";
// Buffer for reading a Slot. MUST be at least 16 bytes.
uint8_t dataBuffer[16];

void setup() {
  Serial.begin(115200);
  while (!Serial);

  if (!initPN532()) {
    Serial.println("PN532 Error: Check wiring or 5V power!");
  }
}
```

```

    while (1);
}

Serial.println("\n--- MAIN MENU ---");
Serial.println("[ r ] Read data from tag");
Serial.println("[ w ] Write global data to tag");
}

void loop() {
    if (Serial.available() > 0) {
        char command = Serial.read();
        // Purge buffer
        while (Serial.available() > 0) { Serial.read(); delay(2); }

        if (command == 'r' || command == 'R') {
            Serial.println("\n>> READ MODE: Place tag on sensor...");

            // Wait for tag
            while (scanForTag() != PN532_OK) { delay(100); }

            if (readTagMemory(TARGET_SLOT, dataBuffer) == PN532_OK) {
                Serial.print(">> Data in Slot 4: [ ");
                for (int i = 0; i < 16; i++)
                    Serial.print((char)dataBuffer[i]);
                Serial.println(" ]");
            } else {
                Serial.println(">> [!] Read Failed.");
            }
            delay(2000);
        }

        else if (command == 'w' || command == 'W') {
            Serial.println("\n>> WRITE MODE: Place tag on sensor...");

            // Wait for tag
            while (scanForTag() != PN532_OK) { delay(100); }

            if (writeTagMemory(TARGET_SLOT, globalData) == PN532_OK) {
                Serial.println(">> [SUCCESS] Data burned to tag.");
            } else {
                Serial.println(">> [!] Write Failed.");
            }
            delay(2000);
        }
    }
}
}

```

 **Try it out:** You can find the full sample program **SimplePollingNFCControllerPN532** in the K-Comp examples folder!